

Coding for Artificial Intelligence

Advanced Function Signatures

Problem Set 7

Positional/Keyword-Only Parameters and Manual Deep-Copy Technique

Project: The Autonomous Payload Packet Parser

Course	Coding for AI (Week 7)
Author	Dr. Prateek, Assistant Professor, School of AI
Format	In-class practical, 1 hour 30 minutes total, inclusive of write-up and submission
Total Marks	100

Read this document in full before you begin writing code.

Contents

1	Objective	2
2	The Problem: The Autonomous Payload Packet Parser	2
3	Required Function Signature	2
4	Clarification: The Role of delimiter	3
5	Implementation Constraints	3
6	Detailed Requirements	3
7	Worked Trace	4
8	Edge Cases to Handle	4
9	Suggested Time Allocation	5
10	Deliverables	5
	Academic Integrity	5

1 Objective

In spacecraft telemetry systems, downlink packets arrive as nested, **mutable** lists containing metadata headers and sensor metrics. A parsing module that accidentally mutates the caller's original packet — rather than working on an independent copy — can silently corrupt mission data. Your objective this week is to build a small, robust, **side-effect-free** telemetry parser, while practising two advanced function-signature features that matter a great deal in real APIs: **positional-only** and **keyword-only** parameters.

By the end of this project you should be able to:

- (i) write a function signature that uses `/` and `*` to control exactly how each parameter may be supplied,
- (ii) build an independent copy of a nested list using slicing inside a comprehension, without importing `copy`, and
- (iii) write verification `assert` statements that prove a function has no side effects on its input.

Time budget. This entire activity — coding, testing, and writing up your submission — is designed to fit in **90 minutes**. A suggested breakdown is given in Section 9.

2 The Problem: The Autonomous Payload Packet Parser

You are given a downlink packet, structured as a list of three elements:

```

1 raw_packet = [packet_id, [sensor_1, sensor_2, ...], status_flag]
2 # packet_id:    int      -- a unique packet identifier
3 # sensor list:  list     -- floating-point sensor readings
4 # status_flag: str      -- one or more short status codes, joined by
5 #                  a delimiter, e.g. "NOMINAL|CALIBRATED"
6 #                  or "ERROR|SENSOR_FAULT"
```

Write a function, `parse_payload`, that returns a corrected, completely independent copy of the sensor readings — without ever modifying the caller's original `raw_packet`.

3 Required Function Signature

```

1 def parse_payload(raw_packet: list, delimiter: str, /, *,
2                  correction_offset: int = 0) -> tuple:
3     ...
```

`raw_packet` and `delimiter` are **positional-only** (everything before the `/`); `correction_offset` is **keyword-only** (everything after the `*`).

Why this matters, beyond syntax practice. These markers are not decoration — they change what is and is not a valid call, and real libraries use them deliberately. Making `raw_packet` and `delimiter` positional-only means their parameter *names* are an implementation detail: the library author is free to rename them in a future version without breaking anyone's code, because no caller was ever allowed to write `raw_packet=...`. Making `correction_offset` keyword-only means every call site *must* write `correction_offset=...` explicitly, which prevents a very real class of bug: silently swapping two same-typed arguments (imagine a fourth, similarly-typed parameter being added later — keyword-only arguments cannot be passed in the wrong position by accident).

Calling `parse_payload` incorrectly should raise a `TypeError` in each of the following cases; you will confirm this yourself in Section 7.

- (i) `parse_payload(raw_packet=my_packet, delimiter="|")` — illegal, because `raw_packet` and `delimiter` cannot be passed by keyword.
- (ii) `parse_payload(my_packet, "|", 2)` — illegal, because `correction_offset` cannot be passed positionally.

4 Clarification: The Role of `delimiter`

The `status_flag` string may contain **one or more** status codes joined by `delimiter`, for example `"NOMINAL|CALIBRATED"` or `"ERROR|SENSOR.FAULT"` (with `delimiter = "|"`). Use `delimiter` to split `status_flag` into its individual codes, and use the result to decide **whether** the correction offset should be applied at all:

- (i) If `"ERROR"` is one of the status codes, the sensor readings are already known to be unreliable. **Do not** apply `correction_offset` to them — correcting drift on data that is already flagged as faulty would misleadingly imply the readings are trustworthy.
- (ii) Otherwise, apply `correction_offset` to every sensor reading as normal.

5 Implementation Constraints

These constraints exist to make sure you understand the underlying mechanics of copying and argument-passing, not to catch you out. Each is explained.

- (i) **No copy or `copy.deepcopy`.** You must build the corrected sensor list using **slicing inside a comprehension** — e.g., iterating over `raw_packet[1][:]` — rather than importing a module that does the copying for you. This is the same underlying technique you used in Problem Set 3 (Section on Memory Integrity): a slice always returns a brand-new list object, so building your result from a sliced, comprehended iteration guarantees the return value shares no structure with the original.
- (ii) **At most three locally-assigned variable names inside `parse_payload`** (parameters do not count toward this limit). This constraint exists to push you toward writing the copy-and-correct logic as a small number of clear expressions, rather than unpacking the packet into many intermediate names. A clean reference solution needs only **two**:

```
1 def parse_payload(raw_packet, delimiter, /, *, correction_offset
2     =0):
3     status_tokens = raw_packet[2].split(delimiter)
4     corrected_sensors = [
5         value if "ERROR" in status_tokens else value +
6         correction_offset
7         for value in raw_packet[1][:]
8     ]
9     return raw_packet[0], corrected_sensors
```

6 Detailed Requirements

- (i) **Deep-copy abstraction.** The returned sensor list must be a completely new object built with slicing inside a comprehension, never a reference to (or a shallow alias of) `raw_packet[1]`.

- (ii) **Telemetry calculation.** Apply `correction_offset` to every sensor value, **unless** the packet's status codes include "ERROR" (Section 4).
- (iii) **Double-unpacking return.** Return exactly a 2-tuple: `(packet_id, corrected_sensor_list)`.
- (iv) **No-side-effect verification.** At the bottom of your script, after calling `parse_payload`, include `assert` statements that confirm `raw_packet`'s sensor list is unchanged, and that the returned list is a genuinely different object (compare with `id()`, not just `==`, since two lists can be *equal in value* while still being two different objects — and it is the *object* that must differ here).

7 Worked Trace

Running a correct implementation against two sample packets, plus the two illegal calls from Section 3, produces the following, verified output:

```

1 Test 1 -> 501 [14.5, 15.0, 13.8]
2 Test 1 side-effect assertions passed.
3 Test 2 -> 502 [9.0, 8.5]
4 Test 2 side-effect assertions passed.
5 Test 3 (default offset) -> 503 [1.0, 2.0]
6 Test 4 passed. TypeError raised as expected: parse_payload() got some
7 positional-only arguments passed as keyword arguments: 'raw_packet,
   delimiter'
8 Test 5 passed. TypeError raised as expected: parse_payload() takes 2
9 positional arguments but 3 were given

```

How to check your own arithmetic. Test 1 uses `raw_packet = [501, [12.5, 13.0, 11.8], "NOMINAL|CALIBRATED"]` with `correction_offset=2`: no "ERROR" code is present, so every reading increases by 2 ($12.5 \rightarrow 14.5$, and so on). Test 2 uses `[502, [9.0, 8.5], "ERROR|SENSOR_FAULT"]` with `correction_offset=5`: because "ERROR" is present, the offset is **not** applied, and the returned values equal the originals — while still being a different list object, which the assertions confirm. Test 3 confirms the default `correction_offset=0` works when the keyword argument is omitted entirely.

8 Edge Cases to Handle

- (i) **A status flag with more than one non-error code**, e.g., "NOMINAL|CALIBRATED|LOW_POWER". Confirm your split-and-check logic still works correctly when there are three or more tokens, not just one or two.
- (ii) **An empty sensor list**, `raw_packet = [504, [], "NOMINAL"]`. Confirm your comprehension simply returns an empty list without error.
- (iii) **`correction_offset` explicitly passed as 0** versus omitted entirely. Both should behave identically — confirm this is actually true of your implementation, since it is easy to accidentally special-case one path and not the other.

9 Suggested Time Allocation

Phase	Description	Duration
Implementation	Signature, deep-copy comprehension, and status-flag logic	40 minutes
Testing	Edge cases from Section 8, plus the two illegal-call tests	30 minutes
Documentation & Submission	Header comment, final write-up	20 minutes
Total		90 minutes

10 Deliverables

- (i) **Source Code:** A single file named `payload_parser.py` containing `parse_payload`, your test packets, and your verification assertions.
- (ii) **Header Documentation:** A comment block at the top of your file explaining, in your own words, what `/` and `*` do in a function signature, and why slicing inside a comprehension is sufficient to guarantee no aliasing with the original packet.
- (iii) **Submission Note:** A short paragraph (100–150 words) describing what happened when you deliberately called `parse_payload` the two illegal ways from Section 3, and quoting the exact error message Python produced in each case.

Academic Integrity

You may discuss approaches with peers during the supervised project session, but the code and documentation you submit must be written independently and must reflect your own understanding. Submissions found to be substantially copied will be treated in accordance with the institute’s academic integrity policy.